



UNIVERSIDADE FEDERAL DO CEARÁ
DEPARTAMENTO DE ENGENHARIA DE TELEINFORMÁTICA
ENGENHARIA DE COMPUTAÇÃO
INTERNET DAS COISAS

Projeto IoT – Entrega Final

SeeSafe

João Manoel – Desenvolvimento back-end

Rafael Gurgel – Processamento de imagem

Víctor Chagas – Desenvolvimento front-end

GLOSSÁRIO

Termo	Definição
IBGE	Instituto Brasileiro de Geografia e Estatística
Háptico	Relativo ao tato ou toque, tátil
Visão Computacional	Área da ciência da computação que estuda métodos para interpretar e analisar imagens e vídeos digitais
Processamento de Imagens	Técnica de manipulação, análise e transformação de imagens digitais
IA	Inteligência Artificial; campo da ciência que desenvolve sistemas capazes de realizar tarefas que requerem inteligência humana
Tempo Real	Resposta ou processamento rápido às mudanças no sistema ou ambiente
JSON	JavaScript Object Notation; formato leve de troca de dados, baseado em texto, fácil de ler e escrever
Amazon AWS	Amazon Web Services; plataforma de serviços em nuvem oferecida pela Amazon
App	Aplicativo; software desenvolvido para executar tarefas específicas, geralmente em dispositivos móveis
Geolocalização	Identificação da localização geográfica de um dispositivo ou usuário, geralmente por GPS ou redes móveis
Giroscópio	Sensor que mede ou mantém a orientação e rotação de um dispositivo
CoAP	(Constrained Application Protocol), ou, em uma tradução aproximada, "Protocolo de Aplicação Restrita" é um protocolo leve para dispositivos IoT com recursos limitados
MiDaS	Técnica de visão computacional que permite gerar mapas de profundidade a partir de imagens monoculares
Kurtosis	Mede o "achatamento" da distribuição das magnitudes do giroscópio durante a janela
Skewness	Mede o grau de "assimetria" na distribuição das magnitudes da aceleração

HISTÓRICO DE REVISÕES

Data	Versão	Descrição	Responsável
29/11	V0	Descrição Informal	<i>João Manoel, Rafael Gurgel, Víctor Chagas</i>
18/12	V1	Coleta de dados e transmissão	<i>João Manoel, Rafael Gurgel, Víctor Chagas</i>
20/01	V2	Ajuste de arquitetura, análise de dados, definição de regras de negócio e aplicação de	<i>João Manoel, Rafael Gurgel,</i>

		modelos de IA	<i>Víctor Chagas</i>
05/02	V3	Análise de dados e utilização de outros modelos de classificação	<i>João Manoel, Rafael Gurgel, Víctor Chagas</i>
21/02	V4	Integração final e testes	<i>João Manoel, Rafael Gurgel, Víctor Chagas</i>

1 INTRODUÇÃO

De acordo com o IBGE, em 2023, o Brasil contava com mais de 6 milhões de pessoas com deficiência visual, sendo aproximadamente 500 mil delas cegas. Para se locomover, essas pessoas geralmente utilizam recursos como cães-guia ou bengalas. Embora eficientes, essas alternativas apresentam limitações, como o alto custo associado aos cães-guia e a incapacidade das bengalas de detectar obstáculos elevados. Nos últimos anos, surgiram soluções tecnológicas que oferecem suporte mais avançado, mas seu custo elevado ainda representa uma barreira para ampla adoção.

Diante disso, propomos desenvolver um sistema de baixo custo que utiliza a câmera do celular para detectar obstáculos e emitir alertas de áudio ou vibração para o usuário. Esse sistema democratizaria o acesso a soluções tecnológicas e contribuiria para uma maior autonomia de pessoas com deficiência visual.

2 DESCRIÇÃO DA APLICAÇÃO

O aplicativo utilizará visão computacional, processando imagens capturadas pela câmera do celular para identificar obstáculos no caminho do usuário. O aplicativo será capaz de analisar o ambiente ao redor em tempo real por meio de mensagens de áudio ou vibração. As funcionalidades previstas incluem:

- Identificação de obstáculos;
- Notificação de perigos no caminho;
- Interface simples e acessível para pessoas com deficiência visual;
- Possibilidade de envio de informações a um responsável.

3 PÚBLICO-ALVO

Pessoas com deficiência visual parcial ou total, que buscam uma ferramenta acessível e eficiente para auxiliar na locomoção diária, especialmente em áreas urbanas. Além disso, o aplicativo atenderá também responsáveis ou cuidadores dessas pessoas, que poderão monitorar informações importantes, como a localização do usuário principal e receber avisos de emergência.

Tabela 01. Aplicações semelhantes

Apps/projetos	Descrição	Diferenças
AIRA	Um óculo equipado com câmera e alto-falante. O usuário assina um plano para poder usar o serviço no qual um profissional acessa a câmera e avisa sobre obstáculos em tempo real.	Necessidade de uma pessoa para fazer a análise do ambiente e emitir os avisos. Custo de assinatura.
Strap Tech	Um colete equipado com câmera e sensores para identificar obstáculos à frente e ao redor do usuário que realiza alertas com atuadores hápticos.	Apenas detecta obstáculos físicos próximos. Sem compartilhamento de informações.
Be My Eyes	Uma pessoa com deficiência solicita auxílio pelo celular, e algum voluntário com o aplicativo é selecionado para realizar a tarefa compartilhando a câmera do dispositivo.	O deficiente necessita que algum voluntário tenha a disponibilidade de aceitar a chamada.
Seeing AI	Utiliza IA para descrever o ambiente ao usuário, incluindo leitura de texto, reconhecimento de objetos e de rostos. Porém, só está disponível em poucos dispositivos e regiões.	Não possui funcionalidades voltadas ao compartilhamento de dados ou monitoramento por um acompanhante.
WeWALK Smart Cane	Bengala inteligente que detecta obstáculos próximos e alerta com sensores hápticos e sonoros. Possui conexão com internet e diversos outros serviços como Google Maps, IA, assistente em tempo real.	Localização apenas em algumas regiões. Custo elevado.

5 DESCRIÇÃO DO SISTEMA

5.1 Requisitos da Aplicação

O aplicativo fornecerá os seguintes serviços para o usuário e o responsável:

- Detecção e notificação de obstáculos em tempo real;
- Possibilidade de operação offline para maior acessibilidade;
- Mensagens de áudio intuitivas para facilitar a compreensão e utilização do aplicativo;
- Envio de mensagens de emergência e localização para o responsável;
- Interface acessível e otimizada para pessoas com deficiência visual.

5.1.1 Requisitos Funcionais

Tabela 02. Requisitos Funcionais da Aplicação

ID	Requisitos funcionais	Descrição
1	Definição de papéis para o usuário	O usuário poderá escolher inicialmente entre "Pessoa com deficiência visual" ou "Monitor/Responsável". Essa seleção determinará as permissões e as funcionalidades disponíveis para cada tipo de usuário, como a visualização de alertas de emergência ou a ativação das funções de assistência ao usuário principal. Também, se o usuário for deficiente visual, deverá ser criado um código que pode ser compartilhado para o monitor/responsável acessar as informações coletadas para aquela sessão de uso.
2	Autenticação	Os dois tipos de usuários devem ser autenticados para estabelecer o fluxo específico de uso para cada usuário, além de identificar as informações no servidor.
3	Processamento de imagem	No fluxo de uso do deficiente visual, o aplicativo deve utilizar a câmera do celular para captar uma (1) imagem a cada X segundos e, em seguida, fazer o processamento dela para extrair dados relevantes. Esses dados serão utilizados para produzir e emitir os alertas de áudio para o usuário.
4	Compartilhamento de informações com o monitor/responsável	Ainda no fluxo de uso do deficiente visual, o aplicativo vai utilizar os dados dos sensores do celular para enviá-los ao fluxo de uso do monitor/responsável via o código de compartilhamento.
5	Exibição de informações de acompanhamento	No fluxo de uso do monitor/responsável, o aplicativo vai utilizar os dados compartilhados via código de sessão para exibir informações relevantes, como localização e alertas de emergência

5.1.2 Requisitos Não-Funcionais

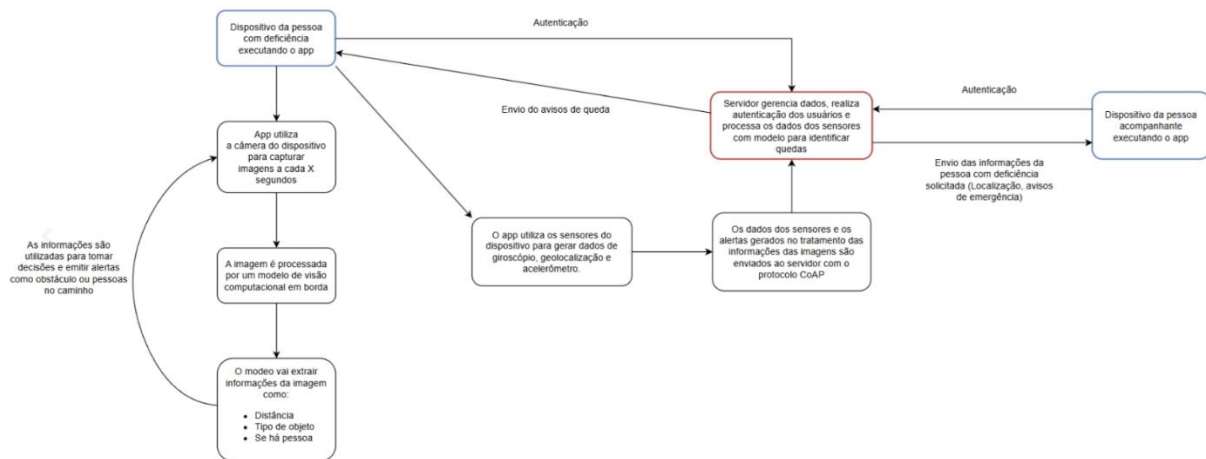
Tabela 03. Requisitos Não-Funcionais da Aplicação

ID	Requisitos Não-Funcionais	Descrição
1	Eficiência	O sistema deve ser estruturado para evitar um consumo elevado de recursos do dispositivo, evitando impactos significativos no

		desempenho do celular.
2	Tempo de resposta	O processamento das imagens deve ser realizado em tempo ágil, visando baixo tempo de resposta, sem causar atrasos ou prejudicar a experiência do usuário.

5.2 Descrição da arquitetura do sistema (pode mudar ao longo do tempo)

Figura 01. Arquitetura do sistema



6 MATERIAIS UTILIZADO

Como o projeto é desenvolvido para dispositivos móveis, estamos utilizando apenas sensores nativos dos aparelhos, como o giroscópio e o acelerômetro, para capturar dados de movimentação e aceleração. Além disso, coletamos também dados de GPS para obter informações de latitude e longitude, bem como capturar imagens com a câmera do celular que serão utilizadas para posterior processamento.

O aplicativo mobile, que realiza a interação entre o usuário e a aplicação, é desenvolvido utilizando os frameworks Expo e React Native, com auxílio da biblioteca de UI Gluestack, o qual realiza a coleta dos dados e envia ao servidor. Por limitações do React Native, algumas partes do aplicativo foram implementadas em código nativo Android (Kotlin), como uma função para utilização do protocolo CoAP de comunicação com o servidor, uma função para captura de imagens e a aplicação do modelo MiDaS com TensorFlow Lite.

O servidor foi desenvolvido em Javascript utilizando o protocolo de comunicação CoAP, além de incorporar, também, um microserviço em Python para execução do modelo de predição. O microserviço foi desenvolvido utilizando FastAPI, um framework de criação de APIs em Python. Sendo o mesmo responsável por processar os dados de giroscópio e acelerômetro, e assim, utilizar o modelo KNN (K-Nearest Neighbors) o qual foi previamente treinado.

Repositório do aplicativo: <https://github.com/victorchgs/seesafe-app>

Repositório do servidor: <https://github.com/victorchgs/seesafe-server>

7 RESULTADOS

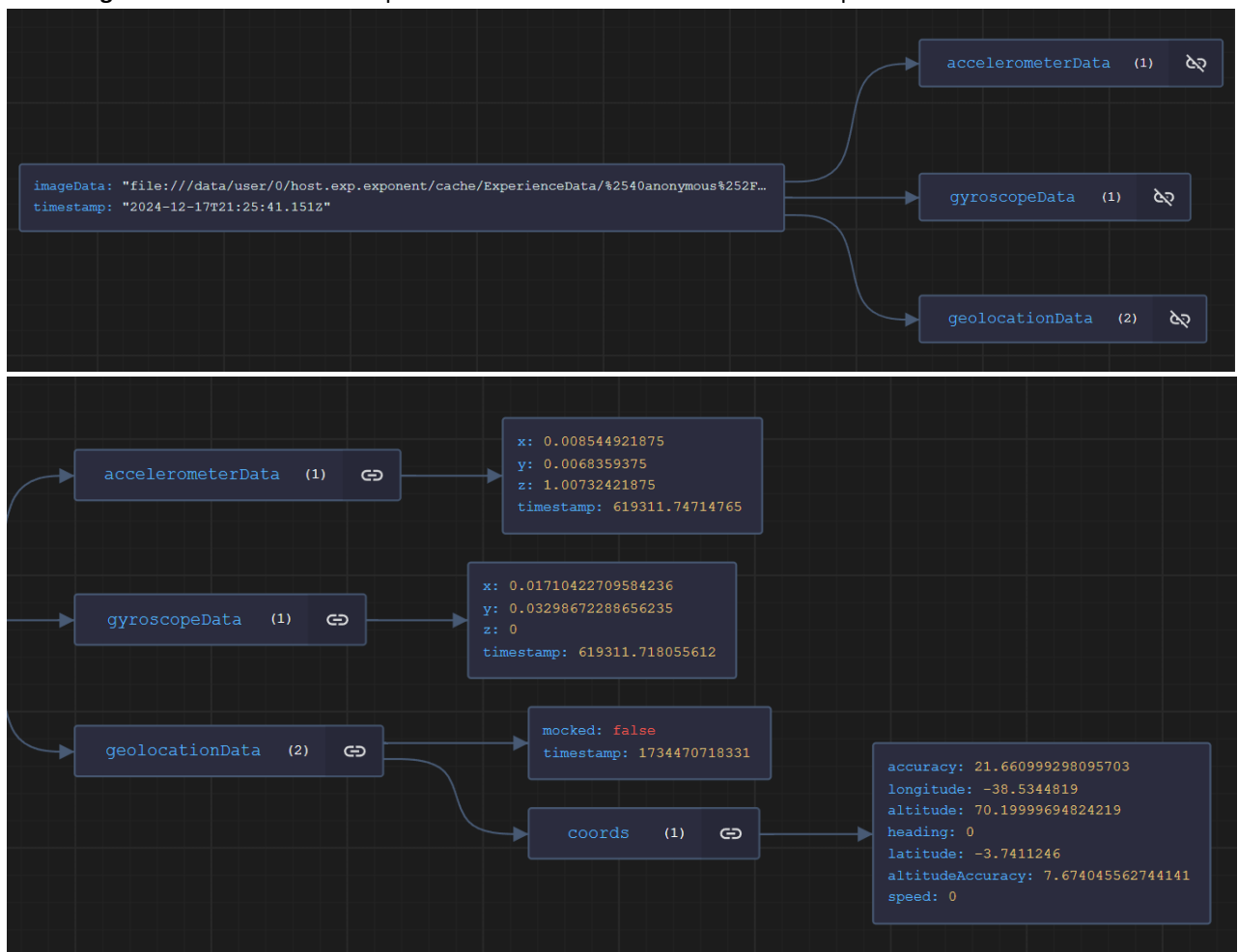
Figura 02. Logs que mostram como vem os dados coleados.

```
(NOBRIDGE) LOG Gyroscope data: {"timestamp": 546164.944488177, "x": -0.006719517521560192, "y": -0.0006108652451075613, "z": -0.012828170321881771}
(NOBRIDGE) LOG Accelerometer data: {"timestamp": 546164.95844983, "x": 0.005615234375, "y": 0.005615234375, "z": 1.01171875}
(NOBRIDGE) LOG Imagem capturada em base64: file:///data/user/0/host.exp.exponent/cache/ExperienceData/%2540anonymous%252Fseesafe-app-7a6a188f-7a28-4d12-b1c3-66139b9a3c16/Camera/e7b940a8-ecad-460c-b6d7-5eb206135389.jpg
(NOBRIDGE) LOG Location data: {"coords": {"accuracy": 20, "altitude": 69, "altitudeAccuracy": 22, "heading": 299.1319274902344, "latitude": -3.7412082, "longitude": -38.5347188, "speed": 0.31519395112991333}, "mocked": false, "timestamp": 1734397576702}
```

Figura 03. Formato em que os dados do usuário são armazenados no Json.

```
id: "id-1734214966574-0.d3yo2mi20kp"
createdAt: "2024-12-14T22:22:46.574Z"
lastAccessed: "2024-12-17T21:25:13.244Z"
status: "active"
```

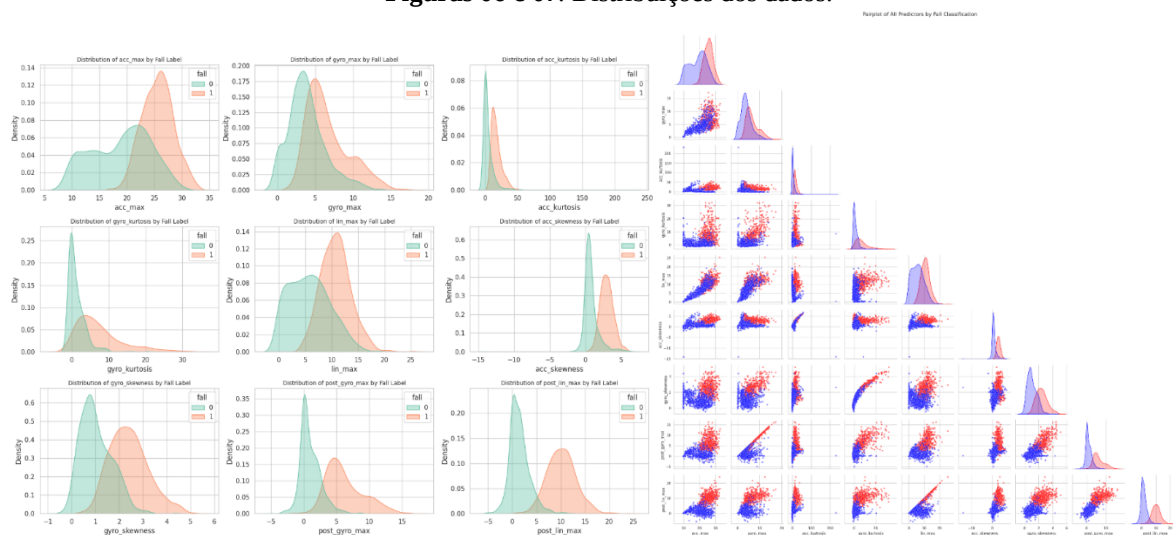
Figura 04 e 05. Formato em que o servidor armazena os dados enviados pelos sensores no JSON.



8.1 Predição de quedas com base em dados de giroscópio e acelerômetro

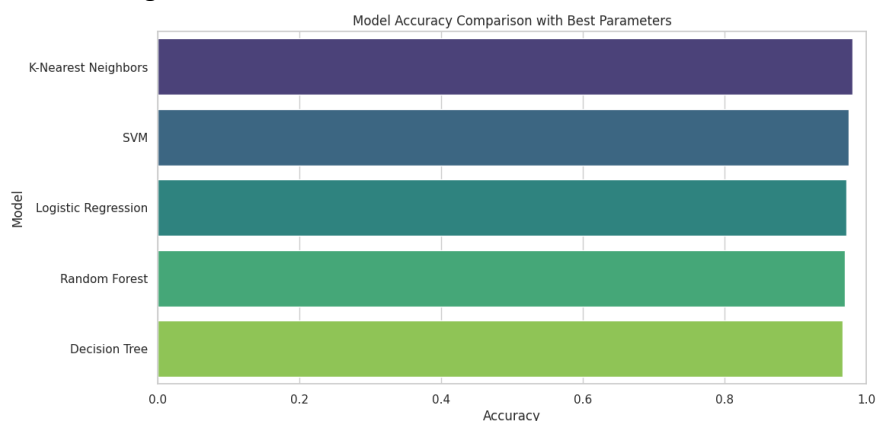
Para realizar a predição foi utilizado a base de dados pública *Smartphone Human Fall Dataset*, a qual tem dados já pré-processados de giroscópio e acelerômetro classificados em queda (1) ou não queda (0), seguintes preditores: Máxima magnitude de aceleração, máxima magnitude de velocidade angular, kurtosis da aceleração, kurtosis da velocidade angular, skewness da aceleração, skewness da velocidade angular, máxima aceleração linear, máxima aceleração linear no último segundo e máxima magnitude do giroscópio no último segundo, onde essas features serão passas ao modelo que classificará quanto a queda ou não queda. Podemos observar nas figuras 06 e 07 quanto a distribuição dos preditores e ouve queda ou não, representado por 0 ou 1.

Figuras 06 e 07. Distribuições dos dados.



A partir dos gráficos de distribuição podemos observar que os dados de queda e não queda possuem certo nível de separação, assim indicando que os modelos de classificação tendem a ter uma acurácia alta no processo de separação de amostras entre queda e não queda. Inicialmente foi realizado uma análise com alguns modelos de classificação, variando seus hiper parâmetros e se obteve os resultados presentes na figura 08, nos quais os modelos KNN e SVM performaram melhor, ficando com uma acurácia de 0,980337 e 0,974719, respectivamente.

Figura 08. Acurácia entre os modelos treinados e testados.



Com isso, utilizando a KNN para realizar o treinamento, podemos observar a matriz de confusão presente na figura 09, a qual indica que o modelo classificou muito bem tanto os dados de queda, como os dados de não queda. Já na figura 10, podemos observar uma série temporal de dados de giroscópio e acelerômetro coletados pela equipe, além disso, também podemos observar no gráfico onde o modelo classificou como sendo queda.

Figura 09. Matriz de confusão.

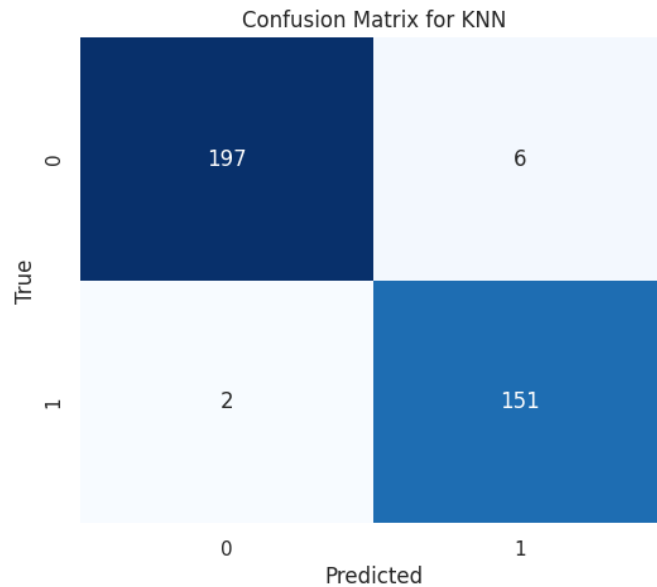
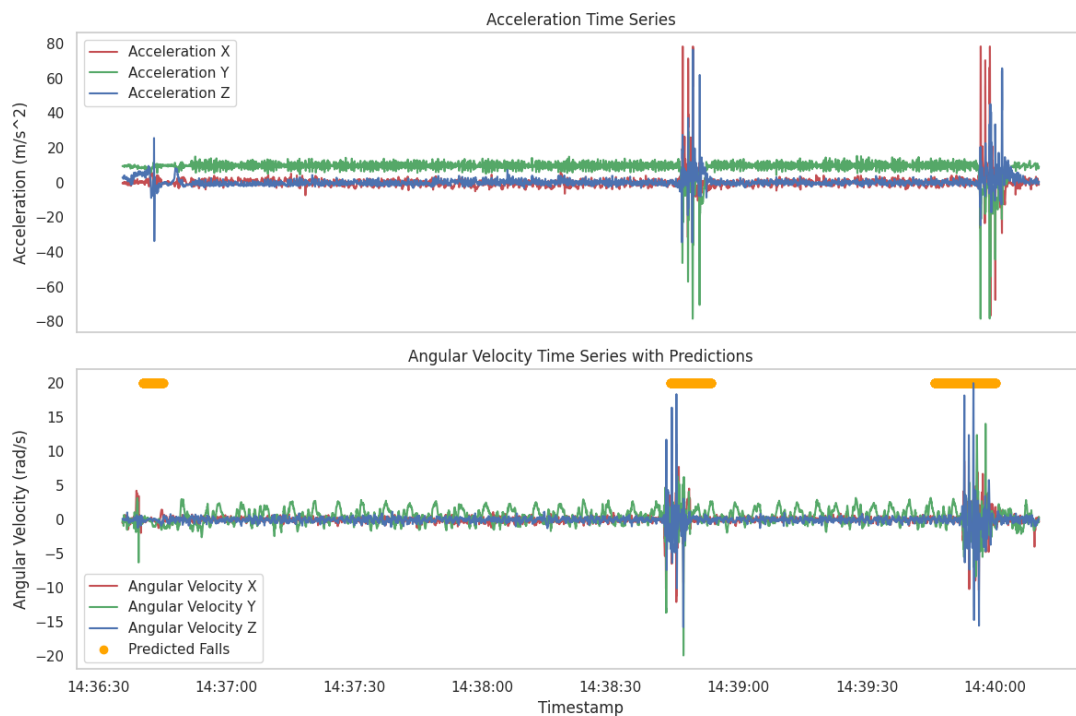


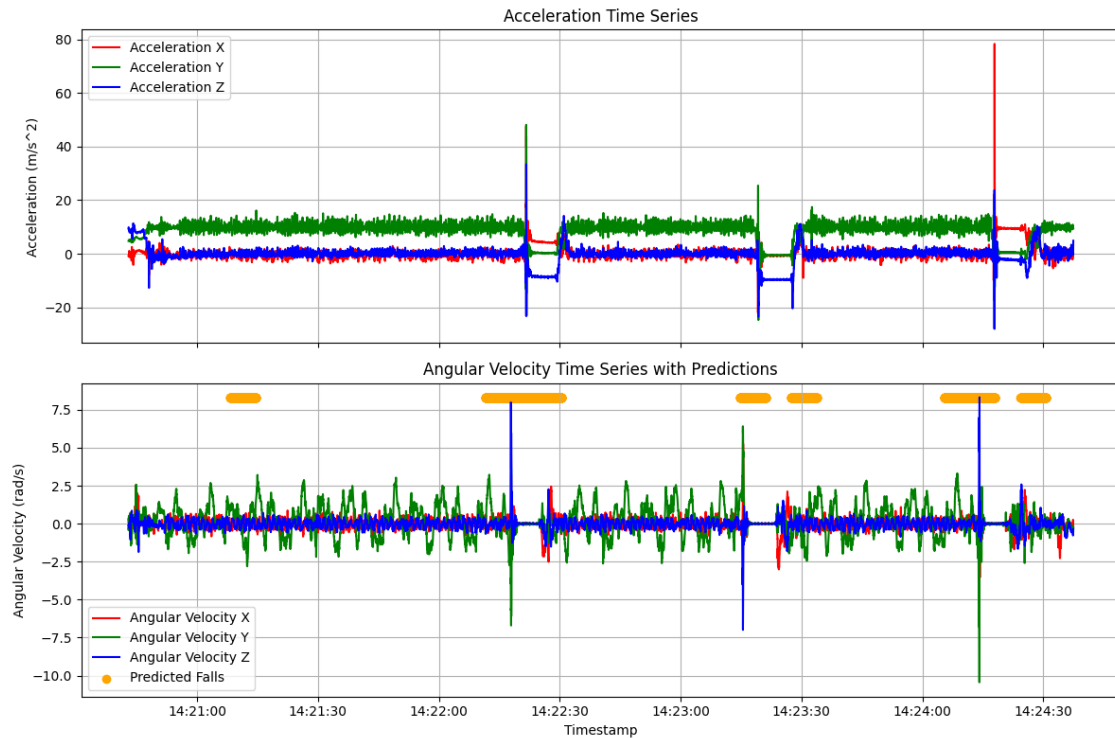
Figura 10. Série temporal com os dados coletados.



Para simular testes de uso real foi simulada uma caminhada, acoplado o smartfone ao corpo do usuário, simulando um processo de caminhada, e em alguns momentos simulando queda de forma mais real, caindo sobre um colchão. No gráfico da figura 11 podemos observar

a simulação real de queda, utilizando o método de classificação KNN, podemos observar que o mesmo obteve um bom desempenho em classificar como queda ou não.

Figura 11. Série temporal com os dados classificados utilizando KNN (simulação real).



8.2 Predição de proximidade de objetos com base no modelo MiDaS

O modelo MiDaS é uma técnica de visão computacional que permite gerar mapas de profundidade a partir de imagens monoculares, ou seja, de imagens contendo apenas uma única câmera. Desenvolvido inicialmente pela equipe de pesquisa da Intel, o MiDaS emprega redes neurais profundas para calcular a profundidade de cada pixel de uma imagem, estimando a distância relativa dos objetos em relação à câmera.

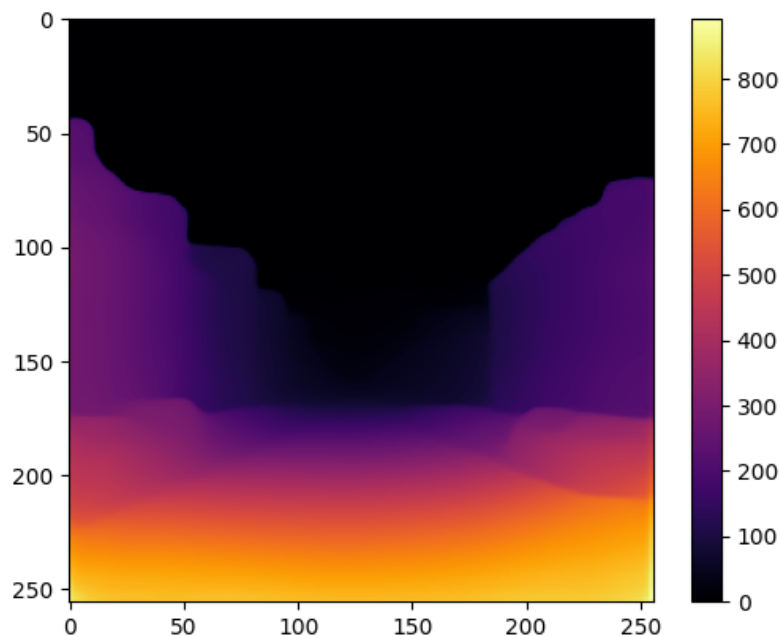
O repositório do MiDaS oferece diversos modelos que atendem a diferentes necessidades, variando desde modelos compactos e rápidos até modelos maiores que proporcionam a mais alta precisão. Esses modelos foram treinados em doze conjuntos de dados distintos, utilizando otimização multiobjetivo, o que garante alta qualidade em uma ampla variedade de entradas e condições (MiDaS, GitHub, 2024).

As imagens 11 e 12 ilustram o funcionamento do modelo MiDaS, a segunda imagem revela como o modelo de profundidade infere as distâncias relativas dos elementos da cena.

Figura 11. Imagem Original



Figura 12. Imagem com Mapa de Profundidade



Uma vez tendo entendido o funcionamento do MiDaS, decidimos utilizar um modelo pré-treinado em TensorFlow Lite, `midas.tflite`, obtido no repositório Kaggle Intel MiDaS Models. Escolhemos esta versão por ser mais otimizada para dispositivos IoT com menos capacidade de processamento, garantindo eficiência e rapidez na estimativa de profundidade. Além disso, inicialmente foi desenvolvida a seguinte regra de negócio para a detecção de objetos próximos:

Detecção de Objetos Próximos:

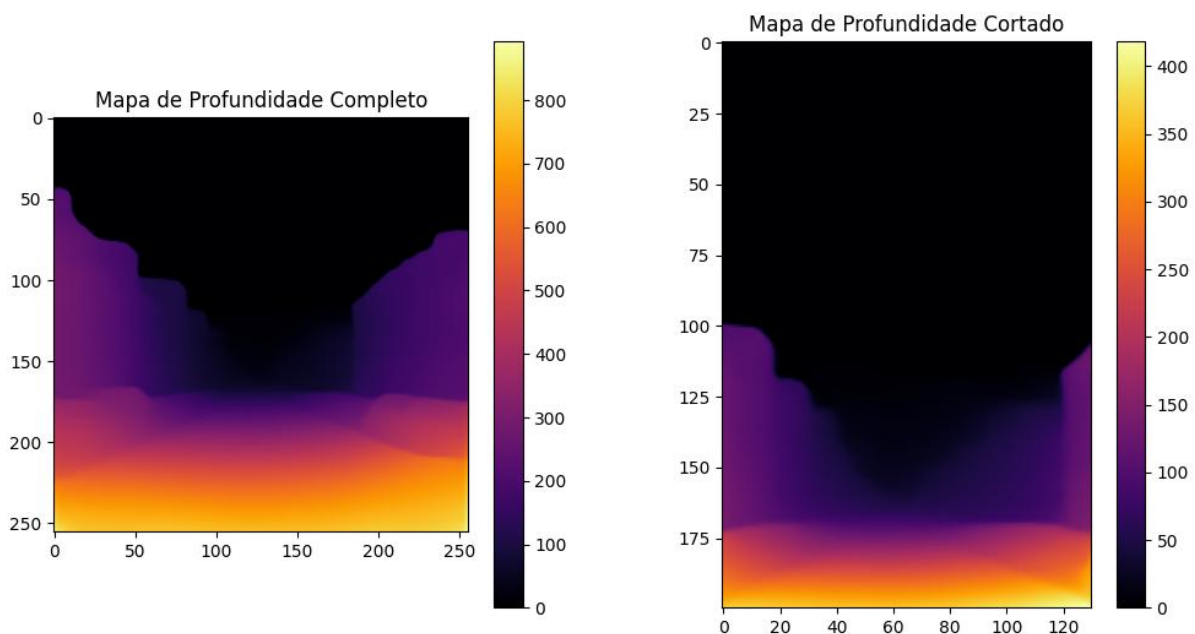
Para determinar a presença de objetos próximos em uma imagem, utilizamos o seguinte processo:

- **Estimativa de Profundidade:** Uma matriz de profundidade é gerada a partir da imagem completa usando o modelo MiDaS.

- **Corte da Região de Interesse:** Selecionamos uma faixa central do mapa de profundidade que representa a área de interesse para a detecção de objetos.
- **Cálculo do Limite de Proximidade:** Determinamos um limiar de proximidade com base em um percentual do valor máximo da matriz de profundidade original.
- **Verificação de Objetos Próximos:** Avaliamos se algum valor na região cortada do mapa de profundidade excede o limiar de proximidade, indicando a presença de um objeto próximo.

Essa decisão foi necessária devido à natureza do modelo monocular MiDaS, que retorna um mapa de profundidade relativo, algumas regiões da imagem – como o chão próximo à câmera – tendem a apresentar valores elevados, o que pode interferir na análise de objetos próximos. Além disso, essa variação relativa faz com que a percepção de obstáculos próximos dependa do contexto da imagem. Assim, o aplicativo tem um desempenho mais consistente em ângulos amplos, onde há maior contraste entre objetos próximos e distantes.

Figura 13. Exemplo da regra de negócio para a detecção de objetos.



8.3 Detecção de obstáculos com base nos modelos YOLO e MiDaS

Figura 14. Exemplo de Classificação com YOLO.

Detecção de Objetos e Obstáculos



Uma segunda abordagem considerada foi a associação do modelo YOLO (You Only Look Once), para a classificação de objetos, ao modelo já explicado MiDaS de estimação de profundidade.

O modelo YOLOv5 foi escolhido, implementado em TensorFlow Lite (tflite), por sua capacidade de detectar e classificar rapidamente objetos em imagens. A ideia central é aplicar o MiDaS para identificar a profundidade nas regiões classificadas como objetos pelo YOLO. O fluxo do processo é descrito a seguir:

- **Detecção de Objetos:** O modelo YOLOv5 tflite é aplicado à imagem para detectar e classificar objetos, gerando caixas delimitadoras (bounding boxes) que identificam a localização de cada objeto na imagem.
- **Estimação de Profundidade:** O modelo MiDaS é então usado para gerar um mapa de profundidade completo da imagem.
- **Extração de Profundidade nas Regiões de Interesse:** As caixas delimitadoras produzidas pelo YOLO são usadas para extrair as regiões correspondentes do mapa de profundidade gerado pelo MiDaS.
- **Cálculo do Limite de Proximidade:** Com base nos valores de profundidade nas regiões de interesse, é determinado um limiar de proximidade para identificar objetos próximos. Este limiar é definido empiricamente.
- **Verificação de Objetos Próximos:** Avaliamos se os valores de profundidade nas regiões de interesse excedem o limiar de proximidade, indicando a presença de obstáculos. Esta abordagem proporciona uma estimativa mais contextualizada das distâncias relativas dos objetos detectados.

8.4 Conclusão sobre as abordagens de detecção de obstáculos

As duas abordagens explicadas nas seções 8.2 e 8.3 apresentam vantagens e desvantagens distintas.

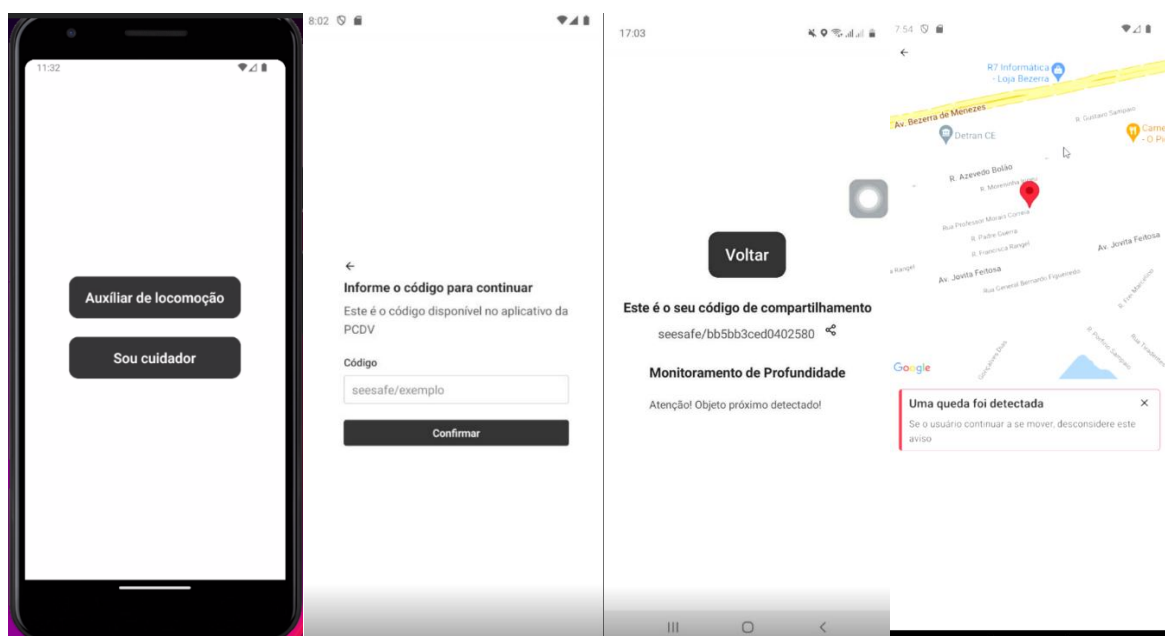
A primeira abordagem, baseada exclusivamente no modelo MiDaS, tem a limitação de restringir a área efetiva de identificação de obstáculos. Isto ocorre devido ao corte da parte central da imagem após a aplicação do MiDaS, o que pode resultar na perda de informações importantes em outras regiões da imagem.

A segunda abordagem, que integra a classificação de objetos pelo modelo YOLO com a estimativa de profundidade do MiDaS, oferece uma detecção mais precisa em áreas específicas onde os objetos foram identificados. No entanto, esta abordagem também apresenta uma limitação significativa: ela só consegue classificar como obstáculos os objetos que são detectáveis pelo modelo YOLO. Isso significa que obstáculos fora do escopo de detecção do YOLO, como paredes ou outros elementos não rotulados, podem ser ignorados.

Dessa forma, optamos por utilizar exclusivamente o modelo MiDaS para a detecção de obstáculos, priorizando uma abordagem que não dependa da classificação prévia de objetos. Apesar da limitação na área efetiva de identificação, essa escolha permite uma estimativa contínua de profundidade em toda a cena, garantindo que obstáculos não rotulados por modelos de detecção de objetos ainda sejam considerados.

9 ATUAÇÃO, INTERFACE E INTEGRAÇÃO

A atuação do dispositivo ocorre tanto na interface do acompanhante quanto no feedback ao usuário. Em caso de queda, o acompanhante conectado recebe uma notificação informando o ocorrido. Além disso, quando um obstáculo é detectado, o aplicativo alerta o usuário com um aviso sonoro, garantindo uma resposta rápida à situação. As figuras a seguir apresentam a versão final das interfaces desenvolvidas.



REFERÊNCIAS OU BIBLIOGRAFIA

1. Kelly Sousa. **Uso de Visão Computacional em Dispositivos Móveis Para auxílio à Travessia de Pedestres com Deficiência Visual**, PPGE, Universidade Presbiteriana Mackenzie, 2013.
2. IBGE aponta que mais de 6 milhões de pessoas têm deficiência visual no Brasil. Disponível em: <https://www.univali.br/noticias/Paginas/ibge-aponta-que-mais-de-6-milhoes-de-pessoas-tem-deficiencia-visual-no-brasil.aspx>. Acesso em: 26 nov. 2024.
3. AIRA. **New Wearables Pilot**. Disponível em: <https://aira.io/new-wearables-pilot/>. Acesso em: 29 nov. 2024.
4. STRAP. **Strap Wearable for the Visually Impaired**. Disponível em: <https://www.strap.tech/>. Acesso em: 29 nov. 2024.
5. BE MY EYES. **Be My Eyes – Connecting Blind and Low-Vision People with Sighted Volunteers**. Disponível em: <https://www.bemyeyes.com/language/portuguese-brazil>. Acesso em: 29 nov. 2024.
6. MICROSOFT. **Seeing AI**. Disponível em: <https://www.seeingai.com/>. Acesso em: 29 nov. 2024.
7. WEWALK. **WeWALK Smart Cane**. Disponível em: <https://wewalk.io/en/>. Acesso em: 2 nov. 2024.
8. **MiDaS**. (2024). GitHub repository. <https://github.com/isl-org/MiDaS>
9. Ranftl, R., Lasinger, K., Hafner, D., Schindler, K., & Koltun, V. **Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer**. arXiv, 2020. Disponível em: <https://arxiv.org/abs/1907.01341>. Acesso em: 21 jan. 2025.
10. VAVOULAS, George; PEDIADITIS, Matthew; SPANAKIS, Emmanouil G.; TSIKNAKIS, Manolis. **Smartphone Human Fall Dataset**. 2022. Disponível em: <https://www.kaggle.com/dsv/4139013>. DOI: 10.34740/KAGGLE/DSV/4139013.
11. MEASURES OF SKEWNESS AND KURTOSIS. **Measures of Skewness and Kurtosis**. 2022. Disponível em: <https://www.itl.nist.gov/div898/handbook/eda/section3/eda35b.htm>. Acesso em: 22 jan. 2025.